

CONFIDENTIAL • STRATEGIC PRODUCT ARCHITECTURE

TECHSPROUT SCHOOL LMS

Product Research, Comprehensive Codebase Audit, REST API & Database Architecture, and Production Launch Guideline

Senior Product Manager: Product Strategy, MVP Scope, EdTech Benchmarks

REST API Architect: Versioned API Envelopes, RBAC & Endpoints

UX / Product Analyst: Learner Portals, Evaluation UX, Mobile Readiness

Software Architect: Modular Monolith, Data Pipelines, Media CDN

Security & SQA Engineer: OWASP Top 10, Auth Hygiene, Test Coverage

Technical Project Manager: Phased WBS, Gantt Roadmaps, Cost Scenarios

Target Platform: Web (Desktop/Mobile) • Future
iOS/Android Native

Document Version: 1.0.0-RC • **Date:**
September 2026

ONE-PAGE EXECUTIVE DECISION SUMMARY

READ FIRST • EXECUTIVE MANDATE

1. What is TechSprout currently?

A polished Next.js 16 frontend marketing prototype. Course browsing, blogs, and landing pages render strictly from in-memory mock data (`src/lib/mockData/mockData.ts`). No real course learning, enrollment, or examination backend exists.

3. What should the MVP contain?

Student/Teacher/Admin accounts, email/phone OTP login, course publishing, modular video player with progress tracking, timed MCQ exams with automatic grading, student dashboard with enrollment history, and single-gateway checkout (bKash/Nagad).

5. What backend architecture is recommended?

A Clean Modular Architecture built inside the Next.js 16 App Router using isolated Domain Services, Repository pattern with Mongoose/MongoDB, and strict DTO validation via Zod schemas. This leverages existing infrastructure while enabling future extraction.

7. What authentication options should be considered?

MVP:

Email/Password + Phone Number with 6-digit SMS OTP + Google OAuth.

Post-MVP:

Facebook Login, passwordless magic links, and session termination across all active devices.

9. What are the biggest risks?

Security:

Plaintext password storage in `/api/register` and open role assignment.

Operational:

Uncontrolled video streaming egress costs without CDN tokens.

Business:

MFS payment dropouts without automated webhook reconciliation.

11. What is the recommended order of development?

1. Security & Auth Stabilization → 2. Role-Based Access Control (RBAC) → 3. Course Hierarchy & Media Hosting → 4. Learning Portal & Progress → 5. Exam/Quiz Engine → 6. Payments & Enrollments → 7. Multilingual → 8. QA Launch Gate.

2. What is missing?

Functional session authentication, student course player/viewer, teacher course builder, admin moderation tools, online examination/quiz engine, certificate generation, real payment gateway (bKash/Nagad/Cards), and internationalization (Bangla support).

4. What should NOT be built yet?

Live Zoom/WebRTC classes, real-time peer chat, AI recommendation engines, gamification token economies, complex proctoring (webcam AI), and native iOS/Android apps prior to stabilizing the core REST API.

6. What REST API architecture is recommended?

A versioned `/api/v1/...` RESTful structure with standardized JSON response envelopes (`{ success, message, data, meta }`), HTTP status codes, bearer JWT sessions, cursor/offset pagination, and centralized error logging.

8. How should Bangla + English work?

Routing via `next-intl` (`/bn/...` , `/en/...`) with cookie fallback; static UI translated via JSON catalogs; course/lesson content stored in bilingual database fields; typography powered by Google Fonts `Hind Siliguri` and `Plus Jakarta Sans` .

10. What is the estimated MVP development timeline?

Solo Developer:

22–26 calendar weeks (approx. 110–130 working days).

Small Team (3 Devs + 1 QA):

10–13 calendar weeks (approx. 50–65 working days).

12. What should I do immediately after this research?

Immediately close the plaintext password vulnerability in `/api/register` , fix the form input binding bug in `Register.tsx` , standardize MongoDB schemas, and implement the formal `/api/v1/auth` service.

Architectural Mandate

Do not write production feature code until the domain models, database schema indexes, and API response contracts detailed in this document are approved. Transitioning an LMS from prototype to production requires absolute data integrity around student enrollments, exam answers, and financial transactions.

Table of Contents

One-Page Executive Decision Summary	Page 2
1. Executive Summary	Section 1
2. Existing Project Audit & Codebase Inspection	Section 2
3. Market & Competitor Research (Bangladesh & Global EdTech)	Section 3
4. Product Vision & Pedagogical Architecture	Section 4
5. MVP Scope Definition (Must Have vs. Should Have vs. Future)	Section 5
6. Launch-Readiness Feature Gap Analysis	Section 6
7. Authentication & Identity Management Strategy	Section 7
8. Language & Internationalization (i18n) Architecture	Section 8
9. Backend Engineering & Services Plan	Section 9
10. REST API Architecture & Formal Specification	Section 10
11. Database Architecture & Data Modeling (MongoDB/Mongoose)	Section 11
12. Online Examination & Assessment Engine	Section 12
13. Evaluation, Grading & Academic Results System	Section 13
14. Student, Teacher & Admin Role Responsibility Matrix	Section 14
15. Security Requirements & Threat Mitigation (OWASP Top 10)	Section 15
16. QA / SQA Launch Audit & Defect Catalog	Section 16
17. Mobile Application Strategy & API Convergence	Section 17
18. Sequential Product Development Roadmap (Phases 1-14)	Section 18
19. MVP Research & Estimated Time Frame to Launch	Section 19
20. Full Product Development Timeline (Solo vs. Small Team)	Section 20
21. Infrastructure, Cloud Resources & Operational Cost Model	Section 21
22. Master Launch Readiness Checklist	Section 22
23. Critical Gap Report (Severity-Ranked Findings)	Section 23
24. Recommended Immediate Next Steps (30-Day Engineering Plan)	Section 24
25. Technical References & Industry Standards	Section 25

01 Executive Summary

TechSprout School is poised to become a premiere online learning management system (LMS) purpose-built for the Bangladeshi ecosystem and regional international learners. The platform aims to bridge the critical gap between academic instruction, practical technical skills, and structured career readiness. To achieve market leadership, TechSprout must deliver a flawless, high-retention digital learning experience characterized by robust video delivery, interactive online examinations, instant academic evaluation, and frictionless localized payments.

This document presents the findings of an exhaustive architectural, codebase, security, and product audit of the current TechSprout repository. While the codebase exhibits modern visual aesthetics using Next.js 16, Tailwind CSS v4, and Radix UI primitives, it is currently in a **pre-alpha prototype state**. The existing implementation lacks core functional backends, exposes serious security vulnerabilities, and utilizes static mock data across all public catalogs.

Strategic Assessment

TechSprout School currently possesses a polished storefront, but **zero operational LMS engines**. Launching the platform requires transitioning from static marketing mockups into an enterprise-grade modular monolith with true data persistence, secure multi-role authentication, automated grading, and compliant MFS billing.

02 Existing Project Audit & Codebase Inspection

Every file, route handler, UI component, database schema, and configuration in `techsprout-main` was inspected. To provide absolute transparency, each core component has been categorized into one of six mutually exclusive technical states:

Status Definitions:

- Fully Implemented** : Complete UI, working business logic, database persistence, and validation.
- Partially Implemented** : Partially working logic or UI; missing error states or edge case handling.
- UI Only / Mock** : Visual interface present, but reads hardcoded data or performs no real actions.
- Broken** : Code exists but causes runtime errors, infinite loops, or severe logic failures.
- Missing** : Feature is required for LMS operations but does not exist anywhere in the codebase.

System Component	Current File / Location	Audit Status	Technical Findings & Code Evidence
Authentication Engine	<code>src/auth.ts</code> <code>src/app/api/[...nextauth]/route.ts</code>	Broken	NextAuth v5 beta is configured, but <code>authorize</code> unconditionally returns <code>null</code> . Logging in is mathematically impossible through NextAuth.

System Component	Current File / Location	Audit Status	Technical Findings & Code Evidence
User Registration API	src/app/api/(user)/register/route.ts	Broken / Critical	Directly persists req.json() to User.create() without password hashing (Argon2/bcrypt) or schema validation. Allows arbitrary role escalation. Returns empty 200 on error.
User Registration Form	src/components/forms/Register.tsx	Broken / Bug	Posts data to scratch endpoint /test instead of /api/register. Confirm Password field is erroneously bound to form key name, overwriting the user's name on typing.
Login Component	src/components/forms/Login.tsx	UI Only / Mock	Submit handler executes console.log(userData). No session initiation, no cookie creation, no redirection to dashboard.
OTP Verification Modal	src/components/OtpModal.tsx src/store/Otpstore.ts	UI Only / Mock	Input OTP slots render, but clicking Submit immediately triggers setModalStatus('close') with no backend verification endpoint, SMS gateway, or email trigger.
Course Catalog & Filtering	src/app/courses/page.tsx	UI Only / Mock	Renders static array courses.slice(0, 6) from mockData.ts. Category checkboxes, price toggles, search input, and pagination buttons have no event handlers.
Course Details & Enrollment	src/app/courses/[course]/page.tsx	UI Only / Mock	Reads synchronous mock data via getSingleCourse(). The "Enroll Now" button is a pure HTML button with no onClick handler or cart integration.
Learning Player & Lessons	src/app/courses/...	Missing	No classroom, video player with curriculum sidebar, lesson completion checklist, or resource download system exists for enrolled learners.

System Component	Current File / Location	Audit Status	Technical Findings & Code Evidence
Online Exam / Quiz System	N/A	Missing	Zero schemas, API endpoints, or UI views exist for creating quizzes, taking timed exams, evaluating questions, or displaying test scores.
Admin Dashboard	src/app/dashboard/admin/courses/page.tsx src/app/dashboard/admin/overview/page.tsx	UI Only / Stub	Consists solely of placeholder headers (e.g., <code><h2>Admin Overview</h2></code>). No course approval tables, analytics, user lists, or financial reports.
Student Dashboard	N/A	Missing	No student portal for enrolled courses, completion certificates, payment receipts, or personal progress tracking.
Instructor Dashboard	N/A	Missing	No instructor tools for managing course curriculums, uploading lecture videos, setting exam questions, or reviewing student grades.
Database Schemas	src/models/*.ts (9 models)	Partially Implemented	Mongoose schemas exist for User, Course, Cart, Enrolment, etc., but are loosely typed (e.g., <code>lessons: [String]</code> instead of subdocuments/references). Lacks indexes and timestamps.
Middleware & Route Guarding	src/middleware.ts	Broken / No-Op	Contains an empty function returning <code>undefined</code> . Protected routes (like <code>/dashboard/*</code>) are completely unprotected and publicly accessible.
Internationalization (i18n)	src/app/layout.tsx	Missing	Root HTML hardcoded to <code>lang="en"</code> with Latin font subsets. No language switcher, translation files, or Bangla font support.
Footer Navigation	src/components/Footer.tsx	Broken Links	Links point to non-existent routes: <code>/about</code> (instead of <code>/about-us</code>), <code>/blog</code>

System Component	Current File / Location	Audit Status	Technical Findings & Code Evidence
			(instead of /blogs), /terms-conditions (instead of /terms-and- condition). Social links lack protocols.

To position TechSprout School competitively, we benchmarked leading platforms in Bangladesh (10 Minute School, Bohubrihi, WebcoachBD) and internationally (Udemy, Coursera). Every finding is strictly classified by evidentiary standard:

Platform	Category / Focus	Core Architecture Findings	Key Lessons for TechSprout
10 Minute School (Bangladesh)	K-12, Admission, Skills	FACT: Mobile-first architecture with OTP phone authentication as the primary sign-in mechanism. FACT: Seamless bKash and Nagad direct merchant API checkout. OBSERVATION: Uses multi-bitrate HLS/DASH video delivery optimized for low-bandwidth cellular networks in rural Bangladesh. RECOMMENDATION: TechSprout must provide phone number OTP sign-in and bKash direct payment for local viability.	Frictionless enrollment via phone numbers is mandatory in Bangladesh. Email-only auth causes up to 60% checkout abandonment.
Bohubrihi (Bangladesh)	Tech & Professional Career Tracks	FACT: Employs structured multi-week project submissions with manual human mentor grading. FACT: Issues verifiable PDF certificates with unique alphanumeric codes and public verification URLs. OBSERVATION: Offers lecture-specific discussion forums where students tag timestamps. RECOMMENDATION: TechSprout should implement automated MCQ grading first, then add mentor project submission post-MVP.	Professional career learners value accredited, verifiable certificates with high academic rigor.
Udemy (Global)	Massive Open Marketplace	FACT: Strict modular hierarchy: Course → Section → Lecture → Resource. OBSERVATION: Video player includes speed adjustments (0.5x–2.0x), timestamped personal notes, and automatic next-lesson triggers. RECOMMENDATION: Adopt Udemy's battle-tested lesson completion model (auto-marking completion upon 90% video watch).	Intuitive, distraction-free video playback with curriculum drawer is the gold standard for student retention.
Coursera (Global)	University & Enterprise Degrees	FACT: Timed summative assessments with strict attempt cooldown periods and randomized question pools. OBSERVATION: Heavy emphasis on peer review evaluation and honor code	Question bank randomization prevents exam answer sharing among student cohorts.

Platform	Category / Focus	Core Architecture Findings	Key Lessons for TechSprout
		enforcement. OPTIONAL FEATURE: Web-based code playgrounds or remote lab containers.	
WebcoachBD (Bangladesh)	Early Tech Tutorials	FACT: Early pioneer of Bengali web development tutorials. OBSERVATION: Suffered from lack of structured evaluation, progress tracking, and monetized interactive exams. RECOMMENDATION: TechSprout must not rely on passive video watching alone; interactive evaluations create sticky engagement.	Pure video repositories face commoditization; interactive LMS platforms provide defensible value.

Synthesized Competitive Strategy

TechSprout School will combine **Udemy's intuitive self-paced learning player** with **10 Minute School's frictionless mobile/bKash onboarding**, backed by **Coursera's rigorous randomized examination engine**, presented in a clean bilingual (Bangla & English) interface.

04 Product Vision & Pedagogical Architecture

The core vision of TechSprout School is to democratize high-caliber technical, vocational, and digital education across Bangladesh. The platform is designed around a three-tier learner progression model:

[1. Conceptual Knowledge] → [2. Hands-on Practice & Quizzes] → [3. Evaluated Assessment & Certification] • High-definition video lectures • In-video knowledge checks • Timed final examinations • Downloadable cheat-sheets • Modular assignments • Cryptographically signed certificates • Bilingual transcripts (BN/EN) • Discussion Q&A with instructor • Public resume verification links

Target User Personas

- **The University Student (Tanvir, 21, Dhaka):** Needs practical programming skills in Bengali. Studies primarily from a smartphone and laptop. Prefers paying via personal bKash wallet.
- **The District Learner (Fatima, 19, Bogura):** Constrained by intermittent broadband. Needs video quality selector (360p–1080p), downloadable lecture notes, and offline quiz caching.
- **The Industry Instructor (Rahim, 32, Senior Software Engineer):** Wants an intuitive course creation suite to upload videos, organize curriculum modules, and generate automated question pools without technical overhead.
- **The Platform Administrator:** Requires comprehensive controls over course moderation, payouts, student disputes, coupon codes, and platform analytics.

05 Minimum Viable Product (MVP) Definition

To avoid scope creep and deliver a stable, revenue-generating product to market, features are strictly partitioned into three tiers:

Feature Category	MVP Tier	Detailed Scope Description	Dependencies	Dev Effort
Dual Authentication	MUST HAVE (MVP)	Email/Password with Argon2id + Bangladeshi Phone Number with 6–digit SMS OTP + Google OAuth.	SMS Gateway (Greenweb/SSL Wireless), NextAuth v5	10 Days
Role-Based Access (RBAC)	MUST HAVE (MVP)	Strict separation between Student, Instructor, and Admin roles via JWT claims and server route guards.	Auth Service	5 Days
Course Curriculum Engine	MUST HAVE (MVP)	Multi-tier hierarchy: Course → Module → Lesson. Support for video streaming, rich text, and PDF attachments.	Mongoose Schemas, S3/Bunny Storage	12 Days

Feature Category	MVP Tier	Detailed Scope Description	Dependencies	Dev Effort
Student Learning Player	MUST HAVE (MVP)	Distraction-free video player with speed control, curriculum progress sidebar, and completion tracking.	Video Hosting CDN, Progress API	10 Days
Automated Quiz Engine	MUST HAVE (MVP)	Single-choice and multi-choice MCQs, randomized question orders, time limits, and instant automated grading.	Exam Service	12 Days
Student Dashboard	MUST HAVE (MVP)	Enrolled courses overview, progress percentages, past quiz scores, and account profile management.	Enrollment & Progress API	8 Days
MFS Payment Integration	MUST HAVE (MVP)	Single payment gateway checkout supporting bKash, Nagad, Rocket, and local debit cards with IPN webhooks.	SSLCommerz / Shurjopay API	9 Days
Automated Certificates	MUST HAVE (MVP)	Dynamic PDF certificate generation upon 100% course + exam completion with unique verification QR/URL.	PDFKit / Puppeteer, Certificate Model	6 Days
Bilingual Support (BN/EN)	MUST HAVE (MVP)	Header language toggle, next-intl dictionary routing, and dual-language titles/descriptions.	next-intl, Translation Files	7 Days
Instructor Portal	SHOULD HAVE (POST-MVP)	Self-service course curriculum editor, video uploader, quiz creator, and enrollment analytics.	Course Engine, File Upload	14 Days
Descriptive Exam Grading	SHOULD HAVE (POST-MVP)	Short-answer and file-submission questions requiring manual instructor review and qualitative feedback.	Exam Engine	8 Days
Discussion Q&A Forum	SHOULD HAVE (POST-MVP)	Lesson-specific threaded discussion board for student-instructor questions.	Forum Schemas	7 Days
Live Video Classes	FUTURE (V2+)	Integrated Zoom / WebRTC live video sessions with calendar reminders.	Livekit / Agora API	18 Days
Native Mobile Apps	FUTURE (V2+)	Cross-platform Flutter or React Native mobile apps with	Stable REST API v1	45 Days

Feature Category	MVP Tier	Detailed Scope Description	Dependencies	Dev Effort
		offline encrypted video caching.		

06 Launch-Readiness Feature Gap Analysis

To establish production readiness, we conducted a granular capability gap analysis comparing expected LMS functionalities against current repository implementations:

Functional Domain	Target LMS Production Requirement	Current TechSprout Implementation	Launch Blockage	Priority
Credential Security	Password hashing via Argon2id or Bcrypt (cost factor 12) + salt.	Plain text JSON stored in <code>User.create()</code> at <code>/api/register</code> .	IMMEDIATE BLOCKER	P0
Form Data Integrity	Password confirmation equality check; strict regex for BD numbers.	<code>Register.tsx</code> binds <code>Confirm Password</code> to <code>name</code> ; posts to <code>/test</code> .	IMMEDIATE BLOCKER	P0
Course Delivery	Protected video streaming (HLS) with signed playback tokens.	Static mock HTML5 <code><video src=demoVideo></code> without token protection.	HIGH BLOCKER	P0
Enrollment Engine	Atomic transaction enrolling user upon verified payment IPN webhook.	"Enroll Now" button has no click handler; no purchase pipeline.	IMMEDIATE BLOCKER	P0
Progress Tracking	Event-driven lesson completion and course percentage calculations.	Zero progress tracking models or endpoints exist.	HIGH BLOCKER	P1
Assessment / Exams	Timed MCQ engine with server-side validation and question randomization.	No exam schemas, questions, or result tables exist.	HIGH BLOCKER	P1
Admin Oversight	Admin portal to approve courses, audit revenues, and manage roles.	Two static pages displaying placeholder <code><h2></code> text.	MODERATE BLOCKER	P1
Bilingual Localization	Full Bengali and English UI with localized number & currency strings.	English-only hardcoded strings; no font support for complex Bangla script.	HIGH BLOCKER	P1

07 Authentication & Identity Management Strategy

A critical comparative benchmark between modern e-commerce/LMS authentication (phone OTP + social login) and TechSprout's current state reveals vital strategic requirements for the Bangladeshi market:

```
[ User Registration / Sign-In Request ] | +-----+-----+ | [ Phone  
Number ] [ Email / Social ] | | Validate BD Regex (+8801...) Check User Table | | Generate 6-Digit OTP  
Verify Argon2id Hash / OAuth Token Store in Redis (TTL: 180s) | Send via SMS Gateway | | Verify OTP  
Code | +-----+-----+ | [ Issue Secure Session ] - HttpOnly Cookie  
(NextAuth JWT) - Refresh Token & Device Fingerprint - Role Claims: Student | Teacher | Admin
```

Authentication Feature Classification

Authentication Method	Classification	Architectural & Market Rationale	Security Controls
Email + Strong Password	REQUIRED (MVP)	Essential for instructors, corporate learners, and administrative personnel who manage formal credentials.	Argon2id hashing, min 8 chars, 1 uppercase, 1 symbol. Zod validation.
Phone Number + 6-Digit OTP	REQUIRED (MVP)	The primary login standard in Bangladesh. Most students use smartphones and lack active email habits.	Redis rate limiting (max 3 OTP requests / 15 mins), 3-minute TTL, constant-time compare.
Google OAuth 2.0	REQUIRED (MVP)	One-tap sign-in reduces friction on desktop browsers and Android smartphones dramatically.	PKCE verification, strict redirect URI validation, automatic email linking.
Facebook Login	RECOMMENDED	Widely utilized in Bangladesh, but frequent API policy shifts and Meta developer review delays make it risky for initial MVP launch.	Strict state token validation to prevent CSRF authentication hijack.
Password Reset via SMS/Email	REQUIRED (MVP)	Without automated password resets, support teams will be overwhelmed with locked account tickets.	Signed, single-use, cryptographically random token with 15-minute expiration.
Session Management & Invalidation	REQUIRED (MVP)	NextAuth JWT with HttpOnly cookies. Support revoking tokens upon role change or password reset.	HttpOnly, Secure, SameSite=Lax cookie flags; token rotation.
Logout from All Devices	RECOMMENDED	Prevents credential sharing where multiple students pool money to access a single paid account.	User <code>tokenVersion</code> integer incremented on global logout.

Authentication Method	Classification	Architectural & Market Rationale	Security Controls
Brute-Force & Abuse Protection	REQUIRED (MVP)	Prevents SMS gateway drainage (financial loss) and credential stuffing attacks.	Upstash Redis sliding-window rate limiting per IP and per phone number.

Language & Internationalization Strategy

To serve Bangladeshi students while maintaining institutional rigor, TechSprout School must support both **বাংলা (Bangla)** and **English** as first-class languages. The architecture must avoid hardcoded string conditionals and establish an extensible i18n foundation:

1. Routing & Locale Persistence Architecture

- **Subpath Routing:** Utilize Next.js 16 with `next-intl` to support canonical localized paths: `techsprout.edu/bn/courses` (Bangla) and `techsprout.edu/en/courses` (English).
- **Default Locale:** Default to `bn` for visitors originating from Bangladesh IP addresses (detected via Cloudflare `CF-IPCountry` header), with cookie fallback (`NEXT_LOCALE=bn`).
- **Language Switcher:** An accessible language toggle in the primary navigation header and mobile drawer, persisting user choice across sessions.

2. Content Translation Strategy

```
[ Content Layer Separation ] | +-----+-----+-----+ | [ Static UI
Chrome ] [ Dynamic Dynamic LMS Content ] • Navigation, buttons, labels • Course titles, descriptions •
Validation & error messages • Lessons, syllabus, exam questions • Handled via JSON Dictionaries • Stored
as localized database subdocuments messages/bn.json , messages/en.json { title: { bn: "...", en: "..."
```

3. Typography, Number & Currency Formatting

- **Bangla Typography:** Google Font **Hind Siliguri** or **Noto Sans Bengali** loaded via `next/font/google` with appropriate font-display swap to eliminate FOIT (Flash of Invisible Text).
- **Numeral Localization:** A centralized utility function converting western digits to Bengali numerals in Bangla mode (e.g., `15` → `১৫`).
- **Currency Standards:** Display course pricing in Bangladeshi Taka (৳ ১,৫০০) for local currency and USD (\$ 20.00) for international visitors.
- **Date & Time:** Formatting dates using native `Intl.DateTimeFormat('bn-BD', { ... })` for natural Bengali calendar displays.

09 Backend Engineering & Services Plan

The backend of TechSprout School will be structured as a **Clean Modular Monolith** built directly on Next.js 16 App Router Route Handlers. This provides serverless scalability, low latency, and zero deployment friction while strictly isolating business domains:

```
[ Client Application / Web UI ] <---( HTTPS JSON )---> [ REST API Route Handlers (/api/v1/...) ] | +-----+
-----+-----+ | | [ Zod DTO Validation ] [ JWT Auth & RBAC Guard ] | | +-----+-----+
| [ Domain Service Layer ] (Auth, Course, Progress, Exam, Payment) | +-----+-----+ | | [
Mongoose Repository ] [ External Cloud Integrations ] | • Cloudflare Stream / Bunny.net [ MongoDB Atlas
Cluster ] • SSLCommerz / bKash Gateway (Replica Sets, Indexes) • SMS Gateway (SSL Wireless) • Email
Dispatch (Resend/SES)
```

Core Backend Architectural Pillars

- **Standardized API Response Envelopes:** All endpoints return a uniform envelope ensuring client predictability:

```
// Success Response (HTTP 200/201)
{
  "success": true,
  "message": "Operation completed successfully",
  "data": { ... },
  "meta": { "page": 1, "limit": 20, "total": 142, "totalPages": 8 }
}

// Error Response (HTTP 400/401/403/404/422/500)
{
  "success": false,
  "message": "Validation failed for one or more fields",
  "errorCode": "VALIDATION_ERROR",
  "errors": [{ "field": "email", "message": "Invalid email address format" }]
}
```

- **Database Transactions:** Atomic multi-document transactions (via `mongoose.startSession()`) for checkout, enrollment, and exam submission to prevent data corruption during network drops.
- **Media Storage Pipeline:** Direct client-to-cloud uploads using presigned S3/Cloudflare R2 URLs for avatars, lecture PDFs, and assignment files. Video lectures ingested through Bunny.net Stream or Cloudflare Stream with automatic HLS transcode.
- **Background Jobs:** Lightweight asynchronous processing using Upstash QStash or BullMQ for sending enrollment emails, SMS OTP alerts, and certificate PDF generation.

10 REST API Architecture & Specification

The REST API is strictly versioned under the `/api/v1` prefix. Below is the proposed endpoint map covering all core domains:

Method	Endpoint Path	Auth / Role	Request Payload	Purpose & Security Rules
POST	<code>/api/v1/auth/register</code>	Public	<code>{ name, phone, password, role? }</code>	Registers student; hashes password with Argon2id; dispatches OTP.
POST	<code>/api/v1/auth/verify-otp</code>	Public	<code>{ phone, otp }</code>	Verifies 6-digit OTP against Redis; marks account verified.
POST	<code>/api/v1/auth/login</code>	Public	<code>{ identifier, password }</code>	Authenticates via phone/email; issues secure HttpOnly JWT.
GET	<code>/api/v1/users/me</code>	Bearer (All)	None	Fetches authenticated user profile, roles, and preferences.
GET	<code>/api/v1/courses</code>	Public	Query: <code>page, limit, category, sort</code>	Returns paginated published courses with price and ratings.
POST	<code>/api/v1/courses</code>	Instructor / Admin	<code>{ title, category, price, ... }</code>	Creates course draft; validates instructor ownership.
GET	<code>/api/v1/courses/:id</code>	Public	None	Returns full course syllabus, instructor bio, and reviews.
GET	<code>/api/v1/courses/:id/learn</code>	Enrolled Student	None	Returns full video stream URLs, lecture notes, and quiz links.
POST	<code>/api/v1/progress/complete</code>	Enrolled Student	<code>{ lessonId, timeWatched }</code>	Records lesson completion; updates overall course progress.
POST	<code>/api/v1/exams/:id/start</code>	Enrolled Student	None	Initiates exam attempt; starts server timer; locks questions.
POST	<code>/api/v1/exams/:id/submit</code>	Enrolled Student	<code>{ attemptId, answers: [...] }</code>	Evaluates answers; computes grade; creates Result record.
POST	<code>/api/v1/payments/initiate</code>	Student	<code>{ courseId, gateway }</code>	Creates pending Order; returns

Method	Endpoint Path	Auth / Role	Request Payload	Purpose & Security Rules
				bKash/SSLCommerz redirect URL.
POST	/api/v1/payments/webhook	Gateway IPN	Gateway signature & payload	Verifies HMAC signature; creates Enrollment; grants access.
GET	/api/v1/certificates/:id	Public	None	Validates authenticity of student completion certificate.

11 Database Architecture & Data Modeling

The persistence layer will utilize **MongoDB with Mongoose** on a high-availability MongoDB Atlas replica set. The data model transitions TechSprout away from loose string arrays to strongly normalized documents with compound indexes, referential integrity, and soft deletion:

```
+-----+ 1:N +-----+ 1:N +-----+ | User | -----> |
Course | -----> | Module | |-----| |-----| |-----| |
_id: ObjectId | | _id: ObjectId | | _id: ObjectId | | name, email, phone | instructor: Ref | |
courseId: Ref | | passwordHash | | title, category | | title, sortOrder | | role: enum | | price, status
| +-----+ +-----+ +-----+ | 1:N | | v | 1:N | 1:N +-----+
-----+ v v | Lesson | +-----+ +-----+ |-----| | Enrollment
| <----- | Order | | _id: ObjectId | |-----| M:1 |-----| |
moduleId: Ref | | userId, courseId | | userId, courseId | | title, type, video | | status, enrolledAt | |
amount, trxId, bK | | duration, sort | +-----+ +-----+ +-----+
| | v v +-----+ +-----+ | CourseProgress | | Exam / Quiz | |-----+
---| |-----| | completedLessons[] | | courseId/lessonId | | percentComplete | | title,
duration | +-----+ | passingMarks | +-----+
```

Key Entity Specifications

Entity Name	Key Fields & Types	Indexes & Constraints	Integrity & Lifecycle Rules
User	_id, name, email, phone, passwordHash, role, isVerified, authProvider	{ email: 1 } (unique, sparse) { phone: 1 } (unique, sparse)	Soft delete with isDeleted: Boolean . Role strictly restricted to ['student', 'instructor', 'admin'] .
Course	_id, title, slug, instructorId, category, price, isPublished, thumbnail, duration	{ slug: 1 } (unique) { instructorId: 1, isPublished: 1 }	Cascade prevents deletion if active enrollments exist. Draft/Published workflow.
Module	_id, courseId, title, sortOrder, isFreePreview	{ courseId: 1, sortOrder: 1 }	Ordered sub-unit. Modules contain one or more sequential Lessons.
Lesson	_id, moduleId, title, type (video/quiz/text), videoStreamId, duration, resources[]	{ moduleId: 1, sortOrder: 1 }	Stores video streaming identifier (e.g. Bunny video ID) rather than raw public MP4 URLs.
Enrollment	_id, userId, courseId, orderId, enrolledAt, expiresAt, status (active/expired)	{ userId: 1, courseId: 1 } (unique)	Atomic creation during payment webhook. Idempotency guarantees access is granted exactly once.
CourseProgress	_id, userId, courseId, completedLessons: [ObjectId],	{ userId: 1, courseId: 1 } (unique)	Updated whenever student completes a lesson.

Entity Name	Key Fields & Types	Indexes & Constraints	Integrity & Lifecycle Rules
	percentComplete, lastAccessedLesson		Triggers certificate generation at 100%.
Exam & Question	_id, title, courseId, durationMinutes, totalMarks, passingScore, questions[]	{ courseId: 1 }	Embedded questions array for high-performance retrieval during student test sessions.
ExamAttempt	_id, examId, userId, startedAt, submittedAt, answers[], score, isPassed	{ userId: 1, examId: 1, startedAt: -1 }	Server calculates score on submission. Immutable audit record of student examination performance.

Online Examination & Assessment Engine

The online examination engine delivers rigorous, tamper-resistant academic testing. It supports formative quizzes (end-of-lesson knowledge checks) and summative final exams (required for course certificates):

1. Question Formats & Randomization

- **Single-Choice MCQ:** Traditional question with 4 options and exactly one correct answer.
- **Multiple-Answer MCQ:** Questions with multiple correct options (partial credit logic applied).
- **True / False:** Rapid conceptual verification questions.
- **Question Pool Randomization:** If an exam requires 20 questions, the server draws randomly from an instructor-curated pool of 50 questions, ensuring adjacent students receive different tests.
- **Option Shuffling:** The order of options (A, B, C, D) is cryptographically shuffled for each attempt.

2. Examination Lifecycle & Integrity Controls

```
[ Student Clicks Start Exam ] | v [ Server Generates Attempt Record ] → Records startedAt timestamp;
Computes hardDeadline = startedAt + duration | v [ Client Displays Questions ] → Client timer countdown
(synced with server via Heartbeat) | +-----+ | v v [ Normal Submission ] [ Hard
Timeout Triggered ] → Server worker automatically auto-submits attempt upon hardDeadline | +-----
-----+ | v [ Server Evaluation Worker ] → Matches answers against answer keys; Applies
negative marking if enabled; Creates Result
```

3. Anti-Cheating & Proctoring Mechanisms (MVP vs. Post-MVP)

- **Full-Screen Lock & Blur Detection (MVP):** Browser `window.onblur` events trigger warning dialogs. Exceeding 3 blur events auto-submits the exam with an academic integrity infraction flag.
- **Copy-Paste & Context Menu Disable (MVP):** CSS and JavaScript restrictions preventing learners from copying question text to search engines.
- **IP / Multiple Device Lock (MVP):** An active exam attempt locks the user session; concurrent logins from another IP address are blocked until the exam completes.
- **Webcam AI Proctoring (Future v2+):** Facial detection, tab monitoring, and periodic snapshots deferred until post-MVP.

13 Evaluation, Grading & Academic Results System

The evaluation system determines learner competency and dictates certificate eligibility. It combines automated computation with instructor moderation:

Grading Algorithms & Negative Marking

- **Instant Scoring:** For objective questions (MCQ, True/False), evaluation executes in under 200ms upon submission. Correct options yield positive points; incorrect choices apply negative marking if designated by the instructor (e.g., -0.25 marks per error to penalize guessing).
- **Passing Score Thresholds:** Courses enforce a configurable passing percentage (default 70%). Students falling below threshold receive actionable breakdown feedback and must observe a 24-hour retake cooldown period.
- **Grading Curves & Letter Grades:** A+ (80–100%), A (70–79%), B (60–69%), C (50–59%), F (<50%).

Certificate Issuance Pipeline

Upon achieving $\geq 70\%$ on the final course exam AND 100% lesson watch completion, the system automatically dispatches an asynchronous job to generate an accredited completion certificate. The PDF features a cryptographically generated SHA-256 hash, student full name, instructor signature, completion date, and an interactive QR code linking to `techsprout.edu/verify/:certificateId`.

14 Student, Teacher & Admin Role Matrix

Access control is strictly enforced via Role-Based Access Control (RBAC). Below is the comprehensive permissions matrix across the three core personas:

Platform Feature / Operation	Student	Instructor / Teacher	Administrator
Browse Public Course Catalog & Syllabus	✓ Allowed	✓ Allowed	✓ Allowed
Enroll in Free & Paid Courses	✓ Allowed	✓ Allowed	✓ Allowed (Bypass Pay)
Watch Paid Course Video Lectures	✓ Enrolled Only	✓ Own Courses	✓ All Courses
Take Quizzes & Final Exams	✓ Enrolled Only	✓ Preview Mode	✓ Preview Mode
Create Course Drafts & Modules	✗ Forbidden	✓ Allowed	✓ Allowed
Publish Course Directly to Public Storefront	✗ Forbidden	✗ Requires Admin Review	✓ Direct Publish
Create Question Pools & Final Exams	✗ Forbidden	✓ Own Courses	✓ All Courses
Grade Subjective Student Assignments	✗ Forbidden	✓ Own Students	✓ Allowed

Platform Feature / Operation	Student	Instructor / Teacher	Administrator
View Course Financials & Royalties	✗ Forbidden	✓ Own Earnings	✓ Platform-Wide
Manage User Accounts & Elevate Roles	✗ Forbidden	✗ Forbidden	✓ Full Control
Issue Refunds & Void Enrollments	✗ Forbidden	✗ Forbidden	✓ Allowed

15 Security Requirements & Threat Mitigation

An enterprise LMS processes sensitive personal identifiable information (PII), proprietary curriculum intellectual property, and financial transactions. Below is the security audit and OWASP Top 10 mitigation framework:

Critical Vulnerabilities Detected in Current TechSprout Repository

- **Plaintext Passwords (CWE-312):** In `src/app/api/(user)/register/route.ts`, `User.create({ ...bodyData })` persists passwords directly to MongoDB without cryptographic hashing. *Remediation: Hash passwords using Argon2id with 64MB memory cost prior to insert.*
- **Mass Assignment & Role Escalation (CWE-915):** The register endpoint accepts unchecked JSON objects. A malicious request specifying `{ "role": "admin" }` immediately gains superuser privileges. *Remediation: Explicit DTO sanitization via Zod schemas, hardcoding new registrations to `role: 'student'`.*
- **Missing Route Guarding (CWE-285):** `src/middleware.ts` is an empty stub. Admin dashboards and internal user data are vulnerable to unauthenticated scraping. *Remediation: Implement NextAuth middleware enforcing valid JWT sessions and role matching.*
- **Unbounded SMS / OTP Abuse (CWE-307):** The OTP mechanism lacks sliding-window rate limiting. Attackers could spam requests, depleting platform SMS gateway funds. *Remediation: Redis rate limiter restricting requests to 3 per phone number per 15 minutes.*

OWASP Top 10 LMS Hardening Guidelines

OWASP Threat Category	LMS Attack Vector	Architectural Countermeasure
A01: Broken Access Control	Student modifies lesson ID in URL to view unpurchased lectures.	Server-side enrollment verification before returning video streaming tokens.
A02: Cryptographic Failures	Cleartext credentials intercepted in transit.	Enforce TLS 1.3, HSTS headers, and Argon2id password hashing with distinct per-user salts.
A03: Injection	NoSQL query injection via malicious MongoDB operators (<code>\$gt</code> , <code>\$or</code>).	Strict Zod schema validation; Mongoose parameterized models; sanitize user input.
A04: Insecure Design	Exam answers sent in client payload and inspected via DevTools.	Answer keys NEVER sent to client. Evaluation performed strictly on server upon submission.
A05: Security Misconfiguration	Exposing stack traces or database connection URLs in error responses.	Sanitize production API errors via <code>errorResponse()</code> ; disable verbose Next.js error overlays.
A07: Identification & Auth Failures	Brute-forcing 6-digit OTP codes or passwords.	Maximum 5 incorrect attempts before invalidating OTP and locking account for 30 minutes.
A08: Software & Data Integrity	Tampered payment webhook payloads granting free course access.	HMAC-SHA256 signature verification on all SSLCommerz and bKash IPN webhooks.

A rigorous quality assurance audit of the active repository identified numerous functional, visual, and linking defects that must be resolved prior to public beta release:

Active Codebase Defect Log

Defect ID	File / Location	Severity	Defect Description & Root Cause
BUG-001	src/components/forms/Register.tsx (Lines 169–170)	CRITICAL	Confirm Password field is bound to form control <code>name="name"</code> . Entering a password confirmation overwrites the user's Full Name.
BUG-002	src/components/forms/Register.tsx (Line 60)	CRITICAL	Submit handler posts user credentials to scratch endpoint <code>/test</code> instead of <code>/api/register</code> .
BUG-003	src/auth.ts (Lines 13–17)	CRITICAL	Credentials provider <code>authorize()</code> returns <code>user = null</code> unconditionally. Authentication always fails.
BUG-004	src/components/Footer.tsx (Lines 24, 32, 44)	HIGH	Dead links to non-existent routes: <code>/about</code> (404), <code>/blog</code> (404), <code>/terms-conditions</code> (404). Correct paths are <code>/about-us</code> , <code>/blogs</code> , <code>/terms-and-condition</code> .
BUG-005	src/components/Footer.tsx (Lines 11–14)	MEDIUM	Social links lack scheme protocols (e.g. <code>www.x.com</code> instead of <code>https://x.com</code>). Causes Next.js router to navigate to relative broken routes.
BUG-006	src/components/header/Header.tsx (Line 282)	HIGH	Mobile navigation sheet renders Register as a standalone <code><Button></code> without an enclosing <code><Link></code> . Clicking Register on mobile is completely non-responsive.
BUG-007	src/components/MouseFollower.tsx (Lines 11–27)	MEDIUM	Animation frame loop is initiated in <code>useEffect</code> without returning a cleanup function to invoke <code>cancelAnimationFrame()</code> . Leads to memory leaks during route transitions.
BUG-008	src/components/forms/Login.tsx (Line 95)	LOW	Password input placeholder reads "Enter you name" instead of "Enter your password". Multiple occurrences of "you" instead of "your" across forms.
BUG-009	src/app/courses/[course]/page.tsx (Line 200)	LOW	Spelling error in course sidebar title: "Recourses" instead of "Resources".

Pre-Launch QA Test Plan & Coverage Matrix

- **Functional Automated Testing (Jest + Playwright):** End-to-end testing for User Registration → OTP Verification → Course Purchase via Sandbox bKash → Lesson Playback → Final Quiz → Certificate Generation.
- **Load & Performance Stress Testing (k6):** Validate that Next.js Route Handlers and MongoDB Atlas can comfortably handle 500 concurrent students taking an exam simultaneously without latency spiking above 500ms.
- **Accessibility (WCAG 2.1 AA):** Ensure color contrast ratios $\geq 4.5:1$ on all text badges, full keyboard tab navigation through curriculum drawers, and proper ARIA labels on all modal dialogs.

17 Mobile Application Strategy & API Convergence

While the immediate focus is web launch, more than 75% of Bangladeshi learners access educational content via Android smartphones. A deliberate mobile convergence strategy prevents architectural duplication:

```
+-----+ | Unified Backend
REST API (/api/v1/...) | +-----+
+-----+ | +-----+ | | [ Web Application
(Next.js 16) ] [ Mobile App (React Native / Flutter) ] • Server-Side Rendering (SEO) • Native Video
Surface (ExoPlayer/AVPlayer) • Desktop Curriculum Management • Offline Encrypted Video Downloads •
Comprehensive Admin & Instructor Dashboards • Biometric Authentication (Fingerprint/FaceID) • Browser
Cookie Session • Push Notifications (FCM / OneSignal)
```

Architectural Rules for Mobile Readiness

- **Stateless REST Contracts:** All `/api/v1/` endpoints must support Authorization Bearer headers in addition to browser cookies, enabling native mobile apps to authenticate without session workarounds.
- **Offline Video DRM & Encryption:** Video downloads on mobile must use AES-128 encrypted HLS segments stored within secure app sandboxes, preventing students from ripping video files.
- **Mobile Exam Experience:** The mobile exam screen must feature large touch-friendly radio targets ($\geq 48\text{px}$ height) and background-pause safeguards in case of incoming cellular phone calls.

Sequential Product Development Roadmap

To ensure engineering rigor, the transformation of TechSprout School is organized into 14 sequential phases. Each phase represents a strict delivery gate:

Phase	Phase Name	Core Deliverables & Engineering Scope	Exit Gate Criteria
Phase 1	Product & Technical Audit	Complete codebase inspection, competitor benchmark, MVP definition, architecture document (Current Phase).	Approved Specification PDF
Phase 2	Security & Core Sanitization	Fix registration bug (BUG-001/002), sanitize schemas, implement Argon2id hashing, clean dead footer links.	Zero Critical Security Flaws
Phase 3	Database & Domain Modeling	Deploy MongoDB Atlas Mongoose models with compound indexes, soft deletes, and automated database seed scripts.	Validated Database Seeds
Phase 4	Authentication & RBAC	Build <code>/api/v1/auth</code> endpoints, Redis OTP sliding-window limiter, SMS gateway integration, NextAuth session guards.	Working Phone OTP & Login
Phase 5	Course & Media Infrastructure	Curriculum schemas (Course → Module → Lesson), Bunny.net video transcode pipeline, presigned S3 upload URLs.	HLS Video Ingest Working
Phase 6	Student Learning Portal	Distraction-free video player, curriculum drawer, lesson completion tracking, and progress percentage computation.	Live Course Player Tested
Phase 7	Examination & Evaluation	Timed MCQ engine, randomized question draws, anti-cheating blur detection, auto-grading, and Results table.	Automated Quiz Grading
Phase 8	MFS Payments & Checkout	SSLCommerz / bKash merchant gateway integration, atomic enrollment transactions, IPN webhook signature verification.	Successful Sandbox Payments
Phase 9	Student & Admin Portals	Student dashboard with active enrollments and quiz scores; Admin moderation dashboard for courses and revenues.	Functional Role Dashboards
Phase 10	Bilingual Localization (i18n)	Integrate <code>next-intl</code> dictionary routing, Google Fonts Hind Siliguri, localized Bengali numbers and Taka currency.	100% Bilingual UI Verified
Phase 11	Accredited Certificates	PDFKit dynamic certificate generation upon 100% completion, cryptographic hash generation, public verification page.	Scannable QR Verification
Phase 12	Comprehensive SQA & Hardening	Automated end-to-end Playwright test suite, k6 stress testing, OWASP Top 10 penetration review.	Zero PO/P1 Defects Remaining

Phase	Phase Name	Core Deliverables & Engineering Scope	Exit Gate Criteria
Phase 13	Production Staging & Launch	Vercel production deployment, MongoDB Atlas production cluster, Cloudflare CDN rules, DNS setup, Public Beta.	Live Production Launch
Phase 14	Native Mobile Apps (v2)	React Native / Flutter mobile app leveraging existing <code>/api/v1/</code> REST endpoints with offline video caching.	Google Play & App Store Release

MVP Research & Estimated Time Frame to Launch

Building a professional, launch-ready LMS requires methodical execution across database design, secure payment handling, and media streaming. The estimated timeline to achieve a public MVP release is detailed below:

1. Minimum Launchable Product (MLP) Definition

The absolute minimum product capable of accepting paying students and delivering educational value must include: **Functional Phone OTP Auth**, **10 Live Video Courses**, **HLS Player with Progress Tracking**, **End-of-Course MCQ Exam**, **bKash/Nagad Checkout**, **Automated PDF Certificate**, and **Bilingual UI Chrome**. Everything else can be added iteratively post-launch.

2. Critical Launch Blockers & Risk Factors

- **Payment Gateway Merchant Onboarding:** Merchant account approval for bKash, Nagad, and SSLCommerz typically takes 10 to 20 business days in Bangladesh due to trade license and KYC compliance verification. *Action: Initiate merchant registration immediately.*
- **SMS Aggregator Sender ID Approval:** Non-masking or masking SMS sender ID approval from BTRC can take 5 to 10 days. *Action: Secure SMS gateway credentials during Phase 2.*
- **Curriculum Video Production:** Technical platforms fail when engineering completes but course content is delayed. At least 5 complete courses must be fully recorded, edited, and encoded prior to public launch.

20 Full Product Development Timeline

To provide realistic planning estimates for stakeholders, we present two staffing scenarios:

Development Phase	Scenario A: Solo Full-Stack Engineer	Scenario B: Small Cross-Functional Team (2 Full-Stack + 1 Frontend + 1 QA)	Key Dependencies
Phases 1–3: Research, Fixes & DB	15 Working Days (3 Weeks)	6 Working Days (1.2 Weeks)	Approved Architecture
Phases 4–5: Auth, RBAC & Media	22 Working Days (4.4 Weeks)	10 Working Days (2 Weeks)	SMS & Bunny.net Accounts
Phases 6–7: Player, Progress & Exams	25 Working Days (5 Weeks)	12 Working Days (2.4 Weeks)	Course & Media Engine
Phases 8–9: Payments & Dashboards	20 Working Days (4 Weeks)	10 Working Days (2 Weeks)	Payment Gateway Sandbox
Phases 10–11: i18n & Certificates	14 Working Days (2.8 Weeks)	7 Working Days (1.4 Weeks)	Translation Catalogs
Phases 12–13: SQA, Staging & Launch	18 Working Days (3.6 Weeks)	9 Working Days (1.8 Weeks)	All Features Deployed
TOTAL ESTIMATED EFFORT	114 Working Days (~23 Calendar Weeks)	54 Working Days (~11 Calendar Weeks)	Ready for Public Beta

Schedule Assumptions

Estimates assume standard 5-day work weeks, timely availability of merchant payment credentials, and no major scope pivots. A small dedicated team reduces calendar time by more than 50% through parallel development of frontend portals, backend services, and test automation suites.

21 Infrastructure, Cloud Resources & Operational Costs

To ensure financial sustainability, the operational infrastructure is modeled across three growth stages: Early MVP (0–500 active students), Growth Stage (500–5,000 active students), and Scale Stage (5,000+ active students):

Resource Category	Recommended Provider	Technical Function	Cost & Scaling Factors
Web & API Hosting	Vercel Pro / DigitalOcean App Platform	Next.js 16 serverless route handlers, Edge middleware, automatic SSL.	Vercel Pro (\$20/dev/mo) handles up to 10k monthly visitors easily.
Managed Database	MongoDB Atlas (Dedicated Tier)	High-availability replica set (M10/M20), automated daily backups, VPC peering.	Free tier (M0) for staging; \$57–\$140/mo for M10 production cluster.
Video CDN & Transcoding	Bunny.net Stream / Cloudflare Stream	Multi-bitrate HLS adaptive streaming, signed DRM tokens, watermark embedding.	\$0.01 per GB egress; approx. \$0.005/min transcoding. Extremely cost-effective.
Document & Asset Storage	Cloudflare R2 / AWS S3	Student assignment submissions, course PDFs, instructor thumbnails.	Cloudflare R2 provides zero egress fees (\$0.015/GB/mo storage).
In-Memory Cache & Limiter	Upstash Redis	OTP code verification, rate limiting, session token invalidation.	Free tier handles 10,000 requests/day; then \$0.20 per 100k requests.
Transactional SMS Gateway	Greenweb BD / SSL Wireless	6-digit OTP delivery to Grameenphone, Banglalink, Robi, Teletalk.	Approx. ₳ 0.25 to ₳ 0.35 BDT per SMS. Masking sender IDs incur setup fee.
Transactional Email	Resend / Amazon SES	Welcome emails, purchase receipts, certificate delivery.	Resend offers 3,000 free emails/mo; then \$20/mo for 50,000 emails.
Payment Gateway Processing	SSLCommerz / bKash Direct Merchant	Processing bKash, Nagad, Rocket, Visa, Mastercard transactions.	bKash merchant fee: 1.2%–1.5%; SSLCommerz MDR: 2.0%–3.5% per successful transaction.

22 Master Launch Readiness Checklist

Every item reflects the verified state of the current codebase. Items marked *Complete* have been verified through active source inspection:

Domain Category	Verification Item	Current Status
Frontend & UI	Responsive landing page, banner, categories, and testimonials	[x] Complete
	Course browsing catalog and search inputs functional	[] In Progress (Mock)
	Course video player with curriculum drawer and progress bar	[] Not Started
Authentication	Registration endpoint with Argon2id password hashing	[!] Needs Review (Broken)
	Phone number with 6–digit SMS OTP verification	[] In Progress (UI Only)
	Role-based route guarding via Next.js middleware	[!] Needs Review (Stub)
Academic Engine	Course & Module schema with database relations	[] In Progress (Loose)
	Online timed MCQ examination and auto-grading engine	[] Not Started
	Automated PDF certificate generation with verification QR	[] Not Started
Monetization	Shopping cart and checkout summary interface	[] In Progress (Cart Stub)
	bKash / Nagad / SSLCommerz IPN webhook integration	[] Not Started
Localization	Bilingual subpath routing (/bn and /en) via next-intl	[] Not Started
	Bangla typography and localized Bengali numerals	[] Not Started
Dashboards	Student portal (enrolled courses, certificates, receipts)	[] Not Started
	Admin moderation panel (approvals, revenues, users)	[] In Progress (Stubs)
QA & Security	OWASP Top 10 hardening and NoSQL sanitization	[] Not Started
	Resolution of all active codebase bugs (BUG-001 to BUG-009)	[!] Needs Review

Critical Gaps Found in Current TechSprout

The following issues represent severe impediments to launching TechSprout School. They are prioritized by security risk, operational failure potential, and user experience impact:

Severity	Identified Problem & Location	Business & Security Impact	Recommended Solution
CRITICAL	Plaintext Password Storage src/app/api/(user)/register/route.ts	Data breach will expose user passwords in clear text, causing catastrophic brand damage and legal liability.	Adopt Argon2id via <code>argon2</code> npm library with salt prior to MongoDB write.
CRITICAL	Mass Assignment / Role Theft src/app/api/(user)/register/route.ts	Any attacker can send <code>role: 'admin'</code> in the registration body and gain full control over the platform.	Strip <code>role</code> from user input; enforce <code>role = 'student'</code> on public signup.
CRITICAL	Register Form Field Overwrite src/components/forms/Register.tsx:169	Typing a confirm password completely overwrites the user's Full Name, preventing successful registration.	Bind form field correctly to <code>name="passwordConfirmation"</code> .
HIGH	Broken Authentication Pipeline src/auth.ts:14	NextAuth <code>authorize()</code> returns <code>null</code> . No student or teacher can log in to access purchased content.	Implement credentials verification against MongoDB password hash.
HIGH	100% Mock Data Dependency src/lib/mockData/mockData.ts	Catalog is static; database updates do not reflect on frontend; dynamic purchases impossible.	Migrate mock arrays to MongoDB collections and query via REST API handlers.
HIGH	Missing Route Middleware src/middleware.ts	Protected pages (admin, student) have no authentication checks and can be	Implement JWT token verification and redirect unauthenticated requests to <code>/login</code> .

Severity	Identified Problem & Location	Business & Security Impact	Recommended Solution
		accessed by any visitor.	
MEDIUM	Dead Links across Footer src/components/Footer.tsx	Visitors clicking About, Blog, or Terms receive jarring 404 error pages, eroding trust.	Fix link hrefs to match existing file system paths (e.g., /about-us).
LOW	Grammatical Inconsistencies Multiple form placeholder inputs	Placeholders read "Enter you email" and "Enter you name", creating an unprofessional impression.	Perform global search-and-replace updating "you" to "your".

Recommended Immediate Next Steps (30-Day Plan)

To transition from planning into high-velocity execution, the engineering team should follow this prioritized 30-day tactical roadmap:

[Days 1 - 5: Critical Bug & Security Patching] • Fix Register.tsx confirm password field binding (BUG-001) • Route registration to /api/v1/auth/register instead of scratch /test • Integrate Argon2id password hashing and Zod input sanitization • Correct dead footer navigation links (BUG-004) | v [Days 6 - 12: Database Schema & Migration] • Implement strict Mongoose schemas (User, Course, Module, Lesson, Enrollment, Exam) • Create database migration script loading mockData.ts into MongoDB Atlas • Establish compound indexes for fast course queries and unique phone/email lookups | v [Days 13 - 20: Authentication & Session Engine] • Complete NextAuth v5 authorize handler with password verification • Build Redis-backed 6-digit phone OTP verification service • Implement route guard middleware protecting /dashboard/* routes | v [Days 21 - 30: Course Catalog API & Player Prototype] • Replace mockApi.ts with real REST API endpoints (/api/v1/courses) • Build functional course learning player UI with module/lesson sidebar • Integrate test video stream via Bunny.net Stream or Cloudflare Stream

Immediate Management Directive

The senior engineering team should immediately create a new feature branch (`feat/security-auth-remediation`) to address BUG-001, BUG-002, and the plaintext password flaw before any further UI styling or marketing efforts take place.

25 Technical References & Industry Standards

The architectural recommendations, security frameworks, and technical standards detailed in this specification are grounded in authoritative industry documentation:

- **OWASP Foundation:** *OWASP Top 10: 2021 (A01: Broken Access Control, A02: Cryptographic Failures, A07: Identification and Authentication Failures)*. Official documentation: owasp.org/Top10.
- **IETF RFC 7519:** *JSON Web Token (JWT) Architecture and Best Current Practices*. Standards Track: [rfc7519](https://tools.ietf.org/html/rfc7519).
- **IETF RFC 9110:** *HTTP Semantics, Status Codes, and REST API Conventions*. Standards Track: [rfc9110](https://tools.ietf.org/html/rfc9110).
- **Next.js 16 Documentation:** *Route Handlers, Server Components, Turbopack, and Middleware Conventions*. Vercel: nextjs.org/docs.
- **W3C Web Accessibility Initiative:** *Web Content Accessibility Guidelines (WCAG) 2.1 AA Standards*. Official recommendation: [w3.org/WAI](https://www.w3.org/WAI).
- **BTRC (Bangladesh Telecommunication Regulatory Commission):** *National Regulatory Directives on A2P SMS Aggregation and Masking Standards*.
- **SSLCommerz & bKash Developer Portals:** *Merchant Integration API Specification v4.0, IPN Webhook Verification, and Tokenized Checkout Guidelines*.